TECHNICAL REPORT · REVISION 2026-08-29

Release Evidence for Deployable AI Models

A contract-based approach to artifact regression, statistical gating, and build localization

MCR is a vendor-neutral, content-addressed evidence envelope. M2RIV is its reference implementation and conformance suite—not a replacement for native model, compiler, or backend oracles.

Abstract

Deployable model releases cross optimizer, compiler, runtime, hardware, registry, and CI boundaries. Native tools can answer local questions while organizations still lack a portable object binding exact artifacts, evidence cohort, statistical interpretation, release policy, and first bad build. This report specifies MCR 1.3 and evaluates it with CPU ONNX cases, cross-language conformance, reference Polygraphy/MLflow integrations, and a live ModelOpt→TensorRT target execution.

24	629×4	100%	build-02
public contracts	GPU comparisons	declared backend matches	first bad

Claim discipline

The GPU result is exact first-party evidence for one RTX 4060/driver/TensorRT/Polygraphy cohort. It is not an independent reproduction, a safety claim, or a cross-hardware performance ranking.

1 · Protocol boundary

Existing system	Native strength	Portable MCR role
MLflow	baseline-aware evaluation and validation	consume verified decision/provenance
Polygraphy	runner output comparison and debug	retain comparator result as evidence
ModelOpt	optimized artifact generation	identify and gate resulting build
TensorRT-Model-Connect	revision/target/perf/quality validation layers	cross-tool handoff semantics

MCR contract

The envelope binds baseline/candidate snapshot IDs, executor/runtime provenance, paired metrics with direction and uncertainty, release policy, explicit authorization, evidence-set and supplemental references, a stable evidence ID, a volatile run ID, and optional ordered-build localization.

Integrity is not authenticity

Strict verification rehashes recognized local components and rejects missing, traversing, symlinked, or inconsistent evidence. A valid bundle is internally complete; it does not identify the producer. The verifier therefore reports self-consistency-only trust unless a separate signing/attestation policy is applied.

Conformance

Producer fixtures cover PASS, WARN, and BLOCK. Consumer receipts must preserve report IDs and statuses and may authorize only PASS. Python/Node/Rust identity vectors and Python<->Rust MCR interop test language neutrality. Repository ownership is disclosed and does not count as adoption.

2 · Experimental design

Element	Declared method
Dataset	UCI handwritten digits via scikit-learn; 1,797 observations
Holdout	seed 23, stratified, 629 cases, unchanged between builds
Weights	reviewed fixed fixture with pinned SHA-256
Critical slice	input-declared digit 1 with normalized ink sum >= 18
Regression	calibration inputs ×1.0 / ×0.65 / ×0.60; weights fixed
Policy	overall margin 3%; critical slice margin 1.5%; paired evidence

CPU ONNX case

The CPU path creates FP16 and QDQ INT8 artifacts with ONNX Runtime. Linux and Windows retain separate numerical/timing evidence while preserving PASS, PASS, BLOCK, BLOCK and build-02 localization. An opset 17→18 control changes graph structure while all declared shared tensors remain equal and emits PASS.

Why the slice matters

The slice is selected from input semantics before candidate outcomes are observed. This avoids post-hoc selection on the dependent variable. In the target run, overall accuracy drops by only 1.59 percentage points at build-02 while the slice drops by 12.77 points.

3 · Live NVIDIA target execution

Cohort	Exact value
GPU	NVIDIA GeForce RTX 4060 Laptop GPU · 8,188 MiB
Software	driver 555.97 · TensorRT 10.4.0 · Polygraphy 0.53.4
Host	Windows build 26200 · AMD64 · Python 3.11.15
Comparison	629 cases · 3 warmups · atol 0.05 · rtol 0.01

Execution chain

Reviewed weights → equivalent PyTorch Conv1d graph → ModelOpt 0.46 INT8 artifacts → TensorRT engines → Polygraphy sequential ONNX Runtime/TensorRT runners → per-output parity evidence → MCR quality gate → monotonic bisect.

Build	Overall	Critical	Parity	Gate
PyTorch FP16	94.75%	91.49%	629/629	PASS
ModelOpt INT8 balanced	94.91%	91.49%	629/629	PASS
ModelOpt INT8 scale .65	93.32%	78.72%	629/629	BLOCK
ModelOpt INT8 scale .60	92.85%	74.47%	629/629	BLOCK

Verification result

First bad = build-02. Four strict MCR bundles verified. Every BackendComparisonEvidence ID was recomputed; unknown local references = 0. Receipt SHA-256: c3d70a68…79490c.

Latency remains run-scoped and is not used here to rank builds. Windows/WDDM did not expose per-process memory through NVML, so VRAM is null with measurement state unavailable—not zero.

4 · Security and failure semantics

Threat	Control	Residual boundary
Poisoned persistent cache	HMAC envelope + key-domain separation	key holders remain trusted
Traversal / symlink evidence	bounded local-root resolution and refusal	host isolation still required
Hostile ONNX	budgets; no custom ops; external data refused	native parser is not a sandbox
Secret exfiltration	metadata blocking + recursive secret canary	operator-defined private endpoints allowed
Incomplete evidence	WARN/ERROR fail closed; strict coverage	authorship needs attestation

Target compatibility is separate

A development observation found an ONNX Runtime QDQ artifact with non-zero zero points rejected by TensorRT 10.4's symmetric-quantization requirement. Parser success, target compatibility, task quality, and performance are distinct claims; one cannot stand in for another.

Corpus state

Case	Status	Expected
ONNX calibration rare-slice regression	verified in CI	BLOCK / build-02
ModelOpt→TensorRT calibration regression	verified on one target	BLOCK / build-02
Recorded rare-slice regression	verified in CI	BLOCK
ONNX opset structural control	verified in CI	PASS

5 · Falsifiability and adoption

The protocol hypothesis is weakened if design partners reject portable promotion semantics or an adjacent standard becomes the accepted equivalent contract. The project must publish these signals instead of hiding them through scope expansion.

Next proof	Why it matters
Independent GPU rerun	separates reproducibility from first-party execution
External producer + consumer	tests whether MCR is a protocol rather than output format
Three retained design-partner gates	tests organizational promotion semantics
Ten independent corpus cases	tests generality across artifact axes

References

MCR specification · docs/mcr-specification.md
MCR conformance · docs/mcr-conformance.md
MLflow evaluation · mlflow.org/docs/latest/ml/evaluation
NVIDIA Polygraphy comparator · docs.nvidia.com/deeplearning/tensorrt/latest/_static/polygraphy/comparator/toc.html
NVIDIA Model Optimizer · github.com/NVIDIA/Model-Optimizer
TensorRT-Model-Connect validation · nvidia.github.io/TensorRT-Model-Connect/user-guides/validate-benchmark/

Conclusion

MCR's claim is narrow and testable: deployable model changes should carry portable, reviewable release evidence across tool boundaries. The reference implementation now demonstrates the complete path from artifact and native backend oracle to strict bundle verification and first-bad localization.